1 Создание фрейма данных

In [1]:
```
import numpy as np
import pandas as pd
```

Ядром pandas являются две структуры данных, в которых происходят все операции: Series и DataFrames.

Series — это объект библиотеки pandas, спроектированный для представления одномерных структур данных, похожих на массивы, но с дополнительными возможностями. Его структура проста, ведь он состоит из двух связанных между собой массивов. Основной содержит данные (данные любого типа NumPy), а в дополнительном, index, хранятся метки.

Создание объекта Series

Для создания объекта Series необходимо вызвать конструктор Series() и передать в качестве аргумента массив, содержащий значения, которые необходимо включить.

In [3]:
```
letters = ['a', 'b', 'c']
numbers = [1, 2, 3]
arr = np.array(numbers)
dict = {'a':1, 'b':2, 'c':3}
```

In [5]:
```
pd.Series(numbers)
```

Out[5]:
```
0    1
1    2
2    3
dtype: int64
```

In [7]:
```
pd.Series(data=numbers, index=letters)
```

Out[7]:
```
a    1
b    2
c    3
dtype: int64
```

In [9]:
```
pd.Series(numbers, letters)
```

Out[9]:
```
a    1
b    2
c    3
dtype: int64
```

In [11]:
```
pd.Series(dict)
```

Out[11]:
```
a    1
b    2
c    3
dtype: int64
```

Если объект Series — аналог одномерного массива с гибкими индексами, объект DataFrame — аналог двумерного массива с гибкими индексами строк и гибкими именами столбцов. Аналогично тому, что двумерный массив можно рассматривать как упорядоченную последовательность выровненных столбцов, объект DataFrame можно рассматривать как упорядоченную последовательность выровненных объектов Series. Под «выровненными» имеется в виду то, что они используют один и тот же индекс.

Создание объекта DataFrame

In [207…
```
df = pd.DataFrame(np.random.randn(4, 5), [1, 2, 3, 4], ['red', 'orange', 'yellow', 'green', '
```

In [209…
```
df
```

Out[209…

	red	orange	yellow	green	blue
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762
3	-0.354465	0.741443	-0.955262	-1.324539	0.510123
4	-0.207300	1.145907	1.600253	0.105940	-0.037480

In [211…
```
df.columns
```

Out[211…
```
Index(['red', 'orange', 'yellow', 'green', 'blue'], dtype='object')
```

In [213…
```
df.index
```

Out[213…
```
Index([1, 2, 3, 4], dtype='int64')
```

In [215…
```
df['orange']
```

Out[215…
```
1   -0.612615
2   -1.848797
3    0.741443
4    1.145907
Name: orange, dtype: float64
```

In [217…
```
type(df)
```

Out[217…
```
pandas.core.frame.DataFrame
```

In [219…
```
type(df['orange'])
```

Out[219…
```
pandas.core.series.Series
```

In [221…
```
df[['yellow', 'blue']]
```

Out[221…

	yellow	blue
1	-1.014738	-0.894796
2	-0.668969	-0.298762
3	-0.955262	0.510123
4	1.600253	-0.037480

In [223…
```
df['indigo'] = df['blue']
```

In [225…
```
df
```

Out[225…

	red	orange	yellow	green	blue	indigo
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762
3	-0.354465	0.741443	-0.955262	-1.324539	0.510123	0.510123
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.037480

In [227…
```
df['violet'] = df['blue'] + df['indigo']
```

In [229…
```
df
```

Out[229…

	red	orange	yellow	green	blue	indigo	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
3	-0.354465	0.741443	-0.955262	-1.324539	0.510123	0.510123	1.020246
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.037480	-0.074960

Навигация по фреймам данных

Для выбора одной или нескольких строк либо значений использовать методы loc или iloc. Все строки во фрейме данных pandas имеют уникальное индексное значение. По умолчанию этот индекс является целым числом, указывающим на положение строки во фрейме данных; однако это не обязательно. Метод loc полезен, когда индекс фрейма является меткой (например, строковым значением). Метод iloc работает, отыскивая позицию во фрейме данных (по номеру индекса).

In [233…
```
df.loc[2]
```

Out[233…
```
red       0.666775
orange   -1.848797
yellow   -0.668969
green     0.015663
blue     -0.298762
indigo   -0.298762
violet   -0.597524
Name: 2, dtype: float64
```

In [235…
```
df.iloc[1]
```

Out[235…
```
red       0.666775
orange   -1.848797
yellow   -0.668969
green     0.015663
blue     -0.298762
indigo   -0.298762
violet   -0.597524
Name: 2, dtype: float64
```

In [237…
```
df.iloc[2] = df.iloc[1]
```

In [239…
```
df
```

Out[239...

	red	orange	yellow	green	blue	indigo	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
3	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.037480	-0.074960

In [241...
```
df.loc[3, 'orange']
```

Out[241...
```
-1.8487966130267144
```

In [243...
```
df.iloc[2, 1]
```

Out[243...
```
-1.8487966130267144
```

In [245...
```
df.loc[[1,3], ['blue', 'violet']]
```

Out[245...

	blue	violet
1	-0.894796	-1.789592
3	-0.298762	-0.597524

Выбор строк на основе условных конструкций

In [248...
```
df[df > 0]
```

Out[248...

	red	orange	yellow	green	blue	indigo	violet
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	0.666775	NaN	NaN	0.015663	NaN	NaN	NaN
3	0.666775	NaN	NaN	0.015663	NaN	NaN	NaN
4	NaN	1.145907	1.600253	0.105940	NaN	NaN	NaN

In [250...
```
df['yellow'] > 0
```

Out[250...
```
1    False
2    False
3    False
4     True
Name: yellow, dtype: bool
```

In [252...
```
df[df['yellow']<0]
```

Out[252...

	red	orange	yellow	green	blue	indigo	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
3	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524

In [254...
```
df[df['blue']<0]['green']
```

Out[254…
```
1   -0.840830
2    0.015663
3    0.015663
4    0.105940
Name: green, dtype: float64
```

In [256…
```
df[df['blue']<0][['green','blue']]
```

Out[256…

	green	blue
1	-0.840830	-0.894796
2	0.015663	-0.298762
3	0.015663	-0.298762
4	0.105940	-0.037480

In [258…
```
df[(df['blue']>0) & (df['yellow']>0)]
```

Out[258…

	red	orange	yellow	green	blue	indigo	violet

In [260…
```
df[(df['blue']>0) | (df['yellow']>0)]
```

Out[260…

	red	orange	yellow	green	blue	indigo	violet
4	-0.2073	1.145907	1.600253	0.10594	-0.03748	-0.03748	-0.07496

Удаление строк

In [263…
```
# удаление строк axis = 0 столбец axis = 1  inplace=False без удаления из датафрейма, inplace=
df.drop(3)
```

Out[263…

	red	orange	yellow	green	blue	indigo	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.037480	-0.074960

In [265…
```
df
```

Out[265…

	red	orange	yellow	green	blue	indigo	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
3	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.037480	-0.074960

In [267…
```
df.drop(3, inplace = True)
```

In [269…
```
df
```

Out[269…

	red	orange	yellow	green	blue	indigo	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.298762	-0.597524
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.037480	-0.074960

In [271…
```
df.drop('indigo', axis = 1, inplace = True)
```

In [273…
```
df
```

Out[273…

	red	orange	yellow	green	blue	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.597524
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960

Удаление повторяющихся строк

Метод drop_duplicates() по умолчанию отбрасывает только те строки, которые идеально совпадают по всем столбцам.

In [277…
```
df.iloc[2]
```

Out[277…
```
red      -0.207300
orange    1.145907
yellow    1.600253
green     0.105940
blue     -0.037480
violet   -0.074960
Name: 4, dtype: float64
```

In [279…
```
row = pd.Series(df.iloc[2])
```

In [281…
```
row
```

Out[281…
```
red      -0.207300
orange    1.145907
yellow    1.600253
green     0.105940
blue     -0.037480
violet   -0.074960
Name: 4, dtype: float64
```

In [283…
```
df
```

Out[283…

	red	orange	yellow	green	blue	violet
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.597524
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960

In [285…
```
df = pd.concat([df, row], ignore_index=0, axis=0)
```

In [287…
```
df
```

Out[287…

	red	orange	yellow	green	blue	violet	4
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907
yellow	NaN	NaN	NaN	NaN	NaN	NaN	1.600253
green	NaN	NaN	NaN	NaN	NaN	NaN	0.105940
blue	NaN	NaN	NaN	NaN	NaN	NaN	-0.037480
violet	NaN	NaN	NaN	NaN	NaN	NaN	-0.074960

In [289…

```
# Удалить дубликаты, показать строки
df.drop_duplicates()
```

Out[289…

	red	orange	yellow	green	blue	violet	4
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907
yellow	NaN	NaN	NaN	NaN	NaN	NaN	1.600253
green	NaN	NaN	NaN	NaN	NaN	NaN	0.105940
blue	NaN	NaN	NaN	NaN	NaN	NaN	-0.037480
violet	NaN	NaN	NaN	NaN	NaN	NaN	-0.074960

In [291…

```
df
```

Out[291…

	red	orange	yellow	green	blue	violet	4
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907
yellow	NaN	NaN	NaN	NaN	NaN	NaN	1.600253
green	NaN	NaN	NaN	NaN	NaN	NaN	0.105940
blue	NaN	NaN	NaN	NaN	NaN	NaN	-0.037480
violet	NaN	NaN	NaN	NaN	NaN	NaN	-0.074960

In [293…

```
df = df.drop_duplicates().head()
```

In [295…
```
df
```

Out[295…

	red	orange	yellow	green	blue	violet	4
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	0.666775	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907

Чтобы выполнить проверку на наличие повторяющихся строк, нередко требуется рассмотреть только подмножество столбцов. Это можно сделать с помощью параметра subset. Метод drop_duplicates по умолчанию сохраняет первое появление повторяющейся строки и удаляет остальные

In [297…
```
df.loc[2,'red'] = df.loc[4,'red']
```

In [299…
```
df
```

Out[299…

	red	orange	yellow	green	blue	violet	4
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907

In [301…
```
df.drop_duplicates(subset=['red'])
```

Out[301…

	red	orange	yellow	green	blue	violet	4
1	-0.478491	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.2073

Замена значений

In [303…
```
df['red'].mean()
```

Out[303…
```
-0.29769717359146103
```

In [307…
```
# Заменить значения
df['red'].replace(df.loc[1, 'red'],df['red'].mean(), inplace=True)
```

In [309…
```
df
```

Out[309…

	red	orange	yellow	green	blue	violet	4
1	-0.297697	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907

Работа с индексами

reset_index() — это метод сброса индекса фрейма данных. Метод .reset_index () задает список целых чисел от 0 до длины данных в качестве индекса.

In [311…

```
df.reset_index()
# inplace = True
```

Out[311…

	index	red	orange	yellow	green	blue	violet	4
0	1	-0.297697	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
1	2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
2	4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
3	red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
4	orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907

In [315…

```
z1 = [2, 3, 4, 5, 6]
df['Z1'] = z1
```

In [317…

```
df
```

Out[317…

	red	orange	yellow	green	blue	violet	4	Z1
1	-0.297697	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN	2
2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN	3
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN	4
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300	5
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907	6

In [321…

```
df.set_index('Z1')
```

Out[321…

Z1	red	orange	yellow	green	blue	violet	4
2	-0.297697	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN
3	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300
6	NaN	NaN	NaN	NaN	NaN	NaN	1.145907

Переименование столбцов

In [323…
```
# Переименовать столбец, показать две строки
df.rename(columns={'red': 'Red'}).head()
```

Out[323…

	Red	orange	yellow	green	blue	violet	4	Z1
1	-0.297697	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN	2
2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN	3
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN	4
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300	5
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907	6

In [325…
```
df
```

Out[325…

	red	orange	yellow	green	blue	violet	4	Z1
1	-0.297697	-0.612615	-1.014738	-0.840830	-0.894796	-1.789592	NaN	2
2	-0.207300	-1.848797	-0.668969	0.015663	-0.298762	-0.597524	NaN	3
4	-0.207300	1.145907	1.600253	0.105940	-0.037480	-0.074960	NaN	4
red	NaN	NaN	NaN	NaN	NaN	NaN	-0.207300	5
orange	NaN	NaN	NaN	NaN	NaN	NaN	1.145907	6

In [ ]:
```
```